



Web Technologies

WET2VO

Teil 10: Medienverarbeitung

Medientechnik und -design | SS 2026

XML-Verarbeitung

XML parsen und schreiben

**Remember
WEF1?**





Parser

XML-Dokumente einlesen und Informationen
programmiertechnisch aufbereiten.

Initialisieren → Aufrufen → Verarbeiten

MARIO
032750

×69

WORLD
4-2

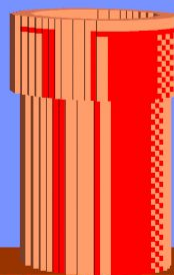
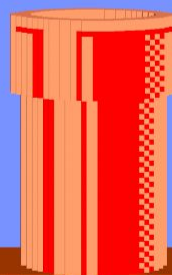
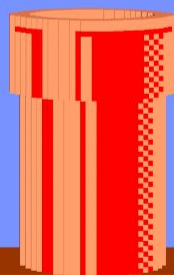
TIME
138

WELCOME TO PARSE ZONE!

EVENT-
BASED

PULL-
PARSE

OO-
PARSE



XML einlesen

Pull-Parser: XMLReader

OO-Parser: DOM



Pull-Parser: XMLReader



```
<?xml version="1.0" encoding="UTF-8"?>
<rezept quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386">
  <gericht>Der klassische Tiroler Knödel</gericht>
  <zutaten>
    <zutat>
      <ingredienz>Semmelwürfel</ingredienz>
      <menge>350</menge>
      <einheit>g</einheit>
    </zutat>
    <zutat>
      <ingredienz>Milch</ingredienz>
      <menge>200</menge>
      <einheit>ml</einheit>
    </zutat>
    ...
  </zutaten>
  ...
</rezept>
```

Diagram illustrating the Pull-Parser (XMLReader) process. A yellow arrow on the left indicates the flow of the XML stream. A pink circle labeled "Knoten" (Node) is connected by pink arrows to the elements being processed: `<gericht>`, `<zutaten>`, and the first `<zutat>` element.

Initialisierung



```
class MyPullParser
{
    private XMLReader $parser;

    private(set) string $source = "";
    private(set) string $dish = "";
    private(set) array $ingredients = [];
    private(set) array $steps = [];

    public function __construct()
    {
        $this->parser = new XMLReader();
    }
}
```

Parsen **starten**



```
public function parse(string $file): void
{
    $this->parser->open($file);

    while ($this->parser->read()) {
        // Verarbeitung ...
    }
}
```

Inhalte verarbeiten



```
while ($this->parser->read()) {  
    if ($this->parser->nodeType === XMLReader::ELEMENT) {  
        switch ($this->parser->name) {  
            case "rezept":  
                $this->source = $this->parser->getAttribute("quelle") ? "": break;  
            case "gericht":  
                $this->dish = trim($this->parser->readString()); break;  
            case "zutat":  
                $this->ingredients[] = []; break;  
            case "ingredienz":  
            case "menge":  
            case "einheit":  
                $lastIndex = array_key_last($this->ingredients);  
                $this->ingredients[$lastIndex][$this->parser->name] =  
                    trim($this->parser->readString()); break;  
            case "schritt":  
                $this->steps[] = trim($this->parser->readString()); break;  
        }  
    }  
}
```

Ablauf **starten**



```
$xmlParser = new MyPullParser();  
$xmlParser->parse("rezept.xml");
```

Ausgabe der ausgelesenen Daten in HTML:

```
<h1><?= $xmlParser->dish ?></h1>  
  
<ul>  
    <?php  
        foreach ($xmlParser->ingredients as $ingredient) {  
            echo "<li>{$ingredient['menge']} {$ingredient['einheit']}  
                {$ingredient['ingredienz']}</li>";  
        }  
    ?>  
</ul>
```

OO-Parser: DOM



```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<rezept quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel">
```

```
  <gericht>Der klassische Tiroler Knödel</gericht>
```

```
  <zutaten>
```

```
    <zutat>
```

```
      <ingredienz>Semmelwürfel</ingredienz>
```

```
      <menge>350</menge>
```

```
      <einheit>g</einheit>
```

```
    </zutat>
```

```
    <zutat>
```

```
      <ingredienz>Milch</ingredienz>
```

```
      <menge>200</menge>
```

```
      <einheit>ml</einheit>
```

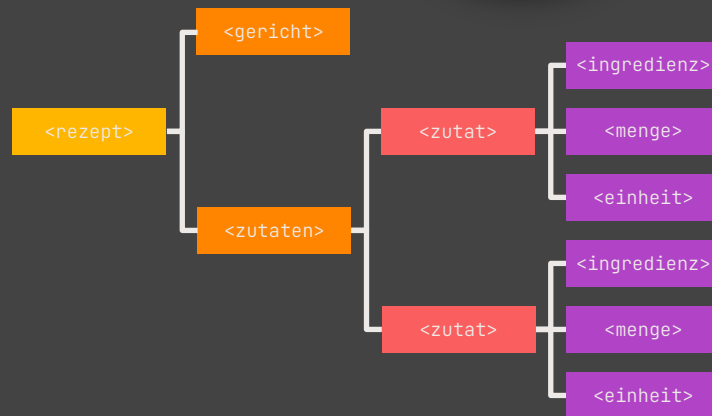
```
    </zutat>
```

```
    ...
```

```
  </zutaten>
```

```
...
```

XML to
DOM
Tree



Initialisierung und Parsen



```
class MyDOMParser
{
    private(set) string $source = "";
    private(set) string $dish = "";
    private(set) array $ingredients = [];
    private(set) array $steps = [];

    public function parse($file)
    {
        $this->source = "";
        $this->dish = "";
        $this->ingredients = [];
        $this->steps = [];

        $dom = XMLDocument::createFromFile($file);

        // Verarbeitung
    }
}
```

Inhalte verarbeiten



```
$this→source = $dom→documentElement→getAttribute("quelle") ? "";

$this→dish = trim($dom→querySelector("gericht")→textContent);

foreach ($dom→querySelectorAll("zutat") as $ingredient) {
    $this→ingredients[] = [];
    foreach ($ingredient→childNodes as $child) {
        if ($child instanceof Element) {
            $lastIndex = array_key_last($this→ingredients);
            $this→ingredients[$lastIndex][$child→nodeName] = trim($child→textContent);
        }
    }
}

foreach ($dom→querySelectorAll("schritt") as $step) {
    $this→steps[] = trim($step→textContent);
}
```

XML erzeugen

XMLWriter
DOM



Ausgangsdaten



```
$shows = [  
  [  
    "name" ⇒ "The Mandalorian",  
    "service" ⇒ "Disney+",  
    "resolution" ⇒ "2160p",  
    "duration" ⇒ "40",  
  ],  
  [  
    "name" ⇒ "Rick and Morty",  
    "service" ⇒ "Netflix",  
    "resolution" ⇒ "1080p",  
    "duration" ⇒ "20",  
  ],  
];
```

XMLWriter: Initialisieren



```
class MyStreamWriter
{
    private XMLWriter $writer;
    private array $shows;

    public function __construct(array $shows)
    {
        $this->writer = new XMLWriter();
        $this->shows = $shows;
    }

    public function generateXML($file)
    {
        $this->writer->openUri($file);
        $this->writer->setIndent(true);
        // Verarbeitung
    }
}
```

XMLWriter: XML erzeugen



```
public function generateXML(string $file): void
{
    // Writer initialisieren
    $this->writer->startDocument("1.0", "UTF-8");
    $this->writer->startElement("shows");

    foreach($this->shows as $show) {
        $this->writer->startElement("show");
        foreach ($show as $tag => $data) {
            $this->writer->writeElement($tag, $data);
        }
        $this->writer->endElement();
    }
    $this->writer->endElement();

    $this->writer->endDocument();
    $this->writer->flush();
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<shows>
  <show>
    <name>The Mandalorian</name>
    <service>Disney+</service>
    <resolution>2160p</resolution>
    <duration>40</duration>
  </show>
  <show>
    <name>Rick and Morty</name>
    <service>Netflix</service>
    <resolution>1080p</resolution>
    <duration>20</duration>
  </show>
</shows>
```

XMLWriter: Ablauf **starten**



```
$shows = [  
    [  
        "name" => "The Mandalorian",  
        "service" => "Disney+",  
        "resolution" => "2160p",  
        "duration" => "40",  
    ],  
    [  
        "name" => "Rick and Morty",  
        "service" => "Netflix",  
        "resolution" => "1080p",  
        "duration" => "20",  
    ],  
];  
  
$xmlWriter = new MyStreamWriter($shows);  
$xmlWriter->generateXML("xmlwriter_shows.xml");
```

DOM: Initialisieren



```
class MyDOMWriter
{
    private array $shows;

    public function __construct(array $shows)
    {
        $this->shows = $shows;
    }
}
```

DOM: XML erzeugen



```
public function generateXML(string $file): void
{
    $dom = XMLDocument::createEmpty("1.0", "UTF-8");
    $dom->formatOutput = true;

    $shows = $dom->appendChild($dom->createElement("shows"));

    foreach($this->shows as $show) {
        $showElem = $shows->appendChild($dom->createElement("show"));
        foreach ($show as $tag => $data) {
            $showData = $dom->createElement($tag);
            $showData->textContent = $data;
            $showElem->appendChild($showData);
        }
    }

    $dom->saveXmlFile($file);
}
```

```
$shows = [
    [
        "name" => "The Mandalorian",
        "service" => "Disney+",
        "resolution" => "2160p",
        "duration" => "40"
    ],
    [
        "name" => "Rick and Morty",
        "service" => "Netflix",
        "resolution" => "1080p",
        "duration" => "20"
    ]
];
```



sabre/xml

```
$ composer require sabre/xml
```

- An XML library for PHP you may not hate.
- Wrapper für XMLReader und XMLWriter
- Einfacherer Zugriff durch Entwurfsmuster

<https://sabre.io/xml/>

sabre-io/xml

sabre/xml is an XML library that you may not hate.



 29
Contributors

 4k
Used by

 526
Stars

 79
Forks



JSON verarbeiten

JSON lesen und schreiben

JSON = Strings



JSON-Funktionen erzeugen Strings und werden aus Strings eingelesen:

```
string json_encode (mixed $value, int $options = 0)
```

```
mixed json_decode (string $json, bool $assoc = false)
```

String in Datei schreiben und aus Datei einen String einlesen:

```
int file_put_contents (string $filename, mixed $data)
```

```
string file_get_contents (string $filename)
```

JSON einlesen



In ein PHP-Objekt oder ein assoziatives Array.

```
mixed json_decode (string $json, bool $assoc = false)
```

Ausgangsdaten



```
{
  "firstName": "John",
  "lastName": "Doe",
  "age": 25,
  "address": {
    "street": "21 Doe St.",
    "city": "Doeville",
    "zip": "12345",
    "state": "NY",
    "country": "USA"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "800 123 456-789"
    },
    {
      "type": "work",
      "number": "800 999 666-333"
    }
  ]
}
```

Lesen als Objekt



```
$jsonString = file_get_contents("addressbook.json");  
$jsonData = json_decode($jsonString);  
var_dump($jsonData);
```

```
/var/www/html/wet2vo-examples/vo10/json/jsondecode.php:11:  
object(stdClass)[1]  
  public 'firstName' ⇒ string 'John' (length=4)  
  public 'lastName' ⇒ string 'Doe' (length=3)  
  public 'age' ⇒ int 25  
  public 'address' ⇒  
    object(stdClass)[2]  
      public 'street' ⇒ string '21 Doe St.' (length=10)  
      public 'city' ⇒ string 'Doeville' (length=8)  
      public 'zip' ⇒ string '12345' (length=5)  
      public 'state' ⇒ string 'NY' (length=2)  
      public 'country' ⇒ string 'USA' (length=3)  
  public 'phoneNumbers' ⇒  
    array (size=2)  
      0 ⇒  
        object(stdClass)[3]  
          public 'type' ⇒ string 'home' (length=4)  
          public 'number' ⇒ string '800 123 456-789' (length=15)  
      1 ⇒  
        object(stdClass)[4]  
          public 'type' ⇒ string 'work' (length=4)  
          public 'number' ⇒ string '800 999 666-333' (length=15)
```

```
{  
  "firstName": "John",  
  "lastName": "Doe",  
  "age": 25,  
  "address": {  
    "street": "21 Doe St.",  
    "city": "Doeville",  
    "zip": "12345",  
    "state": "NY",  
    "country": "USA"  
  },  
  "phoneNumbers": [  
    {  
      "type": "home",  
      "number": "800 123 456-789"  
    },  
    {  
      "type": "work",  
      "number": "800 999 666-333"  
    }  
  ]  
}
```

Lesen als Objekt



```
$jsonString = file_get_contents("addressbook.json");  
$jsonData = json_decode($jsonString);
```

```
echo $jsonData->lastName;  
// Doe
```

```
echo $jsonData->address->street;  
// 21 Doe St.
```

```
echo $jsonData->phoneNumbers[1]->number;  
// 800 999 666-333
```

```
{  
  "firstName": "John",  
  "lastName": "Doe",  
  "age": 25,  
  "address": {  
    "street": "21 Doe St.",  
    "city": "Doeville",  
    "zip": "12345",  
    "state": "NY",  
    "country": "USA"  
  },  
  "phoneNumbers": [  
    {  
      "type": "home",  
      "number": "800 123 456-789"  
    },  
    {  
      "type": "work",  
      "number": "800 999 666-333"  
    }  
  ]  
}
```

Lesen als Array



```
$jsonString = file_get_contents("addressbook.json");  
$jsonData = json_decode($jsonString, true);  
var_dump($jsonData);
```

```
/var/www/html/wet2vo-examples/vo10/json/jsondecode.php:11:
```

```
array (size=5)  
  'firstName' => string 'John' (length=4)  
  'lastName' => string 'Doe' (length=3)  
  'age' => int 25  
  'address' =>  
    array (size=5)  
      'street' => string '21 Doe St.' (length=10)  
      'city' => string 'Doeville' (length=8)  
      'zip' => string '12345' (length=5)  
      'state' => string 'NY' (length=2)  
      'country' => string 'USA' (length=3)  
  'phoneNumbers' =>  
    array (size=2)  
      0 =>  
        array (size=2)  
          'type' => string 'home' (length=4)  
          'number' => string '800 123 456-789' (length=15)  
      1 =>  
        array (size=2)  
          'type' => string 'work' (length=4)  
          'number' => string '800 999 666-333' (length=15)
```

```
{  
  "firstName": "John",  
  "lastName": "Doe",  
  "age": 25,  
  "address": {  
    "street": "21 Doe St.",  
    "city": "Doeville",  
    "zip": "12345",  
    "state": "NY",  
    "country": "USA"  
  },  
  "phoneNumbers": [  
    {  
      "type": "home",  
      "number": "800 123 456-789"  
    },  
    {  
      "type": "work",  
      "number": "800 999 666-333"  
    }  
  ]  
}
```

Lesen als **Array**



```
$jsonString = file_get_contents("addressbook.json");  
$jsonData = json_decode($jsonString, true);
```

```
echo $jsonData["lastName"];  
// Doe
```

```
echo $jsonData["address"]["street"];  
// 21 Doe St.
```

```
echo $jsonData["phoneNumbers"][1]["number"];  
// 800 999 666-333/
```

```
{  
  "firstName": "John",  
  "lastName": "Doe",  
  "age": 25,  
  "address": {  
    "street": "21 Doe St.",  
    "city": "Doeville",  
    "zip": "12345",  
    "state": "NY",  
    "country": "USA"  
  },  
  "phoneNumbers": [  
    {  
      "type": "home",  
      "number": "800 123 456-789"  
    },  
    {  
      "type": "work",  
      "number": "800 999 666-333"  
    }  
  ]  
}
```

JSON erzeugen

Objekte, assoziative Arrays, Arrays, skalare Datentypen.

```
string json_encode (mixed $value, int $options = 0)
```



Ausgangsdaten



```
class Person
{
    public function __construct(
        public string $name,
        public int $age,
        public string $country)
    {}
}
```

```
$array = [
    "Good",
    "Bad",
    "Ugly",
];
```

```
$assocArray = [
    "key" => "value",
    "otherkey" => "othervalue",
];
```

```
$simpleNumber = 42;
```

Constructor Property Promotion



```
class Person
{
    public string $name;
    public int $age;
    public string $country;

    public function __construct(
        string $name,
        int $age,
        string $country)
    {
        $this->name = $name;
        $this->age = $age;
        $this->country = $country;
    }
}
```

Daten schreiben



```
$jsonObject = json_encode(new Person("John Doe", 34, "DoeLand"),  
                           JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT);
```

```
$jsonArray = json_encode($array, JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT);
```

```
$jsonAssocArray = json_encode($assocArray, JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT);
```

```
$jsonInteger = json_encode($simpleNumber, JSON_UNESCAPED_SLASHES|JSON_PRETTY_PRINT);
```

In Dateien schreiben:

```
file_put_contents("object.json", $jsonObject);
```

```
file_put_contents("array.json", $jsonArray);
```

```
file_put_contents("assocarray.json", $jsonAssocArray);
```

```
file_put_contents("integer.json", $jsonInteger);
```

Ergebnisse



object.json

```
{  
  "name": "John Doe",  
  "age": 34,  
  "country": "Doeland"  
}
```

array.json

```
[  
  "Good",  
  "Bad",  
  "Ugly"  
]
```

assocarray.json

```
{  
  "key": "value",  
  "otherkey": "othervalue"  
}
```

integer.json

```
42
```

Grafiken mit GDlib

Erzeugen, konvertieren, skalieren

GD Graphics Library

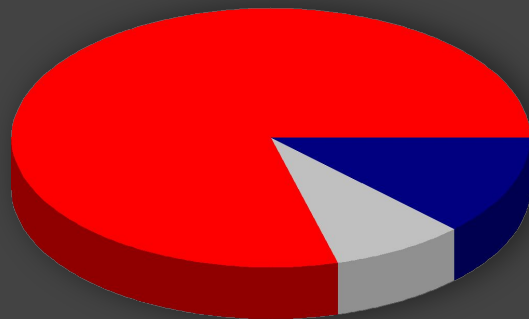


Grafikbibliothek zur Erstellung und Bearbeitung von Bildern.
Seit PHP 4.3.0 fixer Bestandteil von PHP.
<https://libgd.github.io/>



Einsatzzwecke

- Erzeugung von Grafiken
- Formatumwandlung
- Skalierung
- Rotieren



Erstellen & Öffnen

Neue (leere) Bilder erstellen oder bestehende Formate öffnen.



Leere Bilder erstellen



200 x 200 Pixel; 256 Farben

```
$image = imagecreate(200, 200);
```

200 x 200 Pixel; 16,7M Farben

```
$image = imagecreatetruecolor(200, 200);
```

Ergebnis:

`$image` ist eine Zeichenfläche ohne Inhalte (schwarz)

Bestehende Bilder öffnen



```
$image = imagecreatefrompng($filename);  
$image = imagecreatefromjpeg($filename);  
$image = imagecreatefromgif($filename);  
$image = imagecreatefromwebp($filename);  
$image = imagecreatefromavif($filename);  
$image = imagecreatefromtga($filename);  
$image = imagecreatefrombmp($filename);  
$image = imagecreatefromwbmp($filename);  
$image = imagecreatefromxbm($filename);  
$image = imagecreatefromxpm($filename);  
$image = imagecreatefromgd($filename);  
$image = imagecreatefromgd2($filename);
```

Bilder **universell** öffnen



Bild aus Datenstrom (String) mit Bildinhalten erzeugen:

```
$image = imagecreatefromstring($datastream);
```

Datenstrom (String) aus Datei erzeugen:

```
$datastream = file_get_contents($filename);
```

Beides kombiniert – funktioniert für alle unterstützten Dateitypen!

```
$image = imagecreatefromstring(file_get_contents($filename));
```

Bilder ausgeben

Im Browser anzeigen oder in einer Datei am
Server speichern.



Ausgeben/speichern



```
imagepng($image [, $filename]);  
imagejpeg($image [, $filename]);  
imagegif($image [, $filename]);  
imagewebp($image [, $filename]);  
imageavif($image, [, $filename]);  
imagebmp($image [, $filename]);  
imagewbmp($image [, $filename]);  
imagexbm($image [, $filename]);  
imagegd($image [, $filename]);  
imagegd2($image [, $filename]);
```

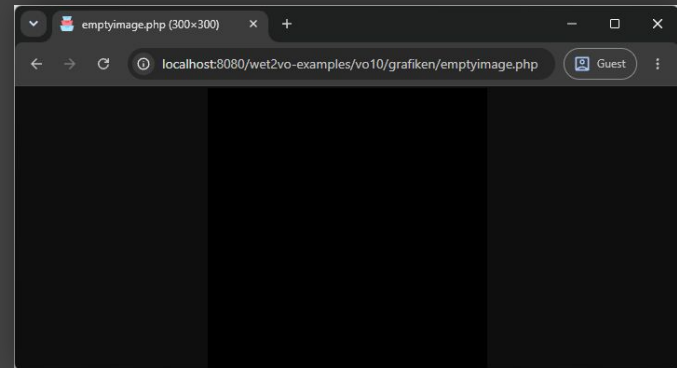
Im **Browser** anzeigen



```
$image = imagecreatetruecolor(300, 300);
```

```
header("Content-type: image/png");
```

```
imagepng($image);
```



In einer **Datei** speichern



```
$image = imagecreatetruecolor(300, 300);
```

```
imagepng($image, "emptyimage.png");
```

Zeichenfunktionen

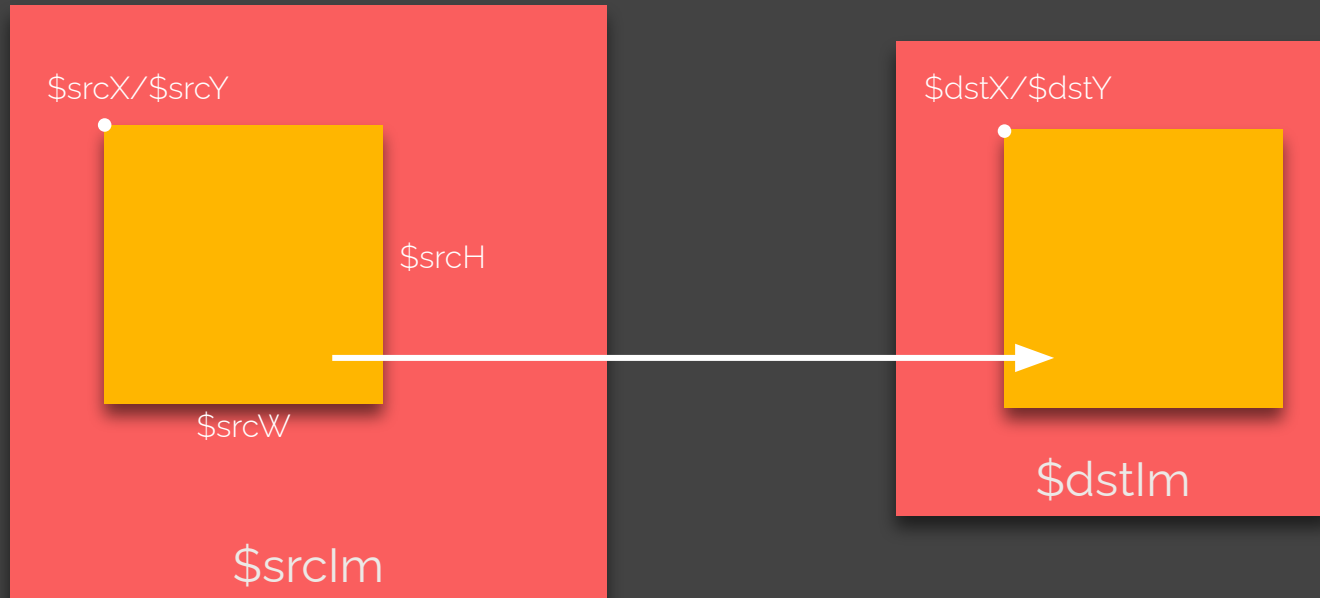


- Farben definieren: `imagecolorallocate($image, $red, $green, $blue)`
- Linienstärke: `imagesetthickness($image, $thickness)`
- Linienstil: `imagesetstyle($image, $style)`
- Einzelne Pixel: `imagesetpixel($image, $x, $y, $color)`
- Linien: `imageline($image, $x1, $y1, $x2, $y2, $color)`
- Rechtecke: `imagerectangle($image, $x1, $y1, $x2, $y2)`
- Gefüllte Rechtecke: `imagefilledrectangle($image, $x1, $y1, $x2, $y2)`
- Polygone: `imagepolygon($image, $points, $color)`
- Kreisbögen: `imagearc($image, $cx, $cy, $w, $h, $s, $e, $c)`
- Ellipsen: `imageellipse($image, $cx, $cy, $w, $h, $c)`
- Text mit TrueType-Schriften: `imagettftext($image, $size, $angle, $x, $y, $color, $font, $text)`
- uvm.

Bilder kopieren



```
imagecopy($dstIm, $srcIm, $dstX, $dstY, $srcX, $srcY, $srcW, $srcH);
```



Ausschnitt kopieren

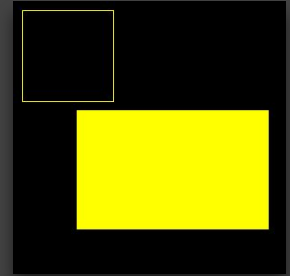


```
$image1 = imagecreatetruecolor(300, 300);  
$image2 = imagecreatetruecolor(100, 100);  
$yellow = imagecolorallocate($image1, 255, 255, 0);
```

```
imagerectangle($image1, 10, 10, 110, 110, $yellow);  
imagefilledrectangle($image1, 70, 120, 280, 250, $yellow);
```

```
imagecopy($image2, $image1, 0, 0, 50, 50, 100, 100);
```

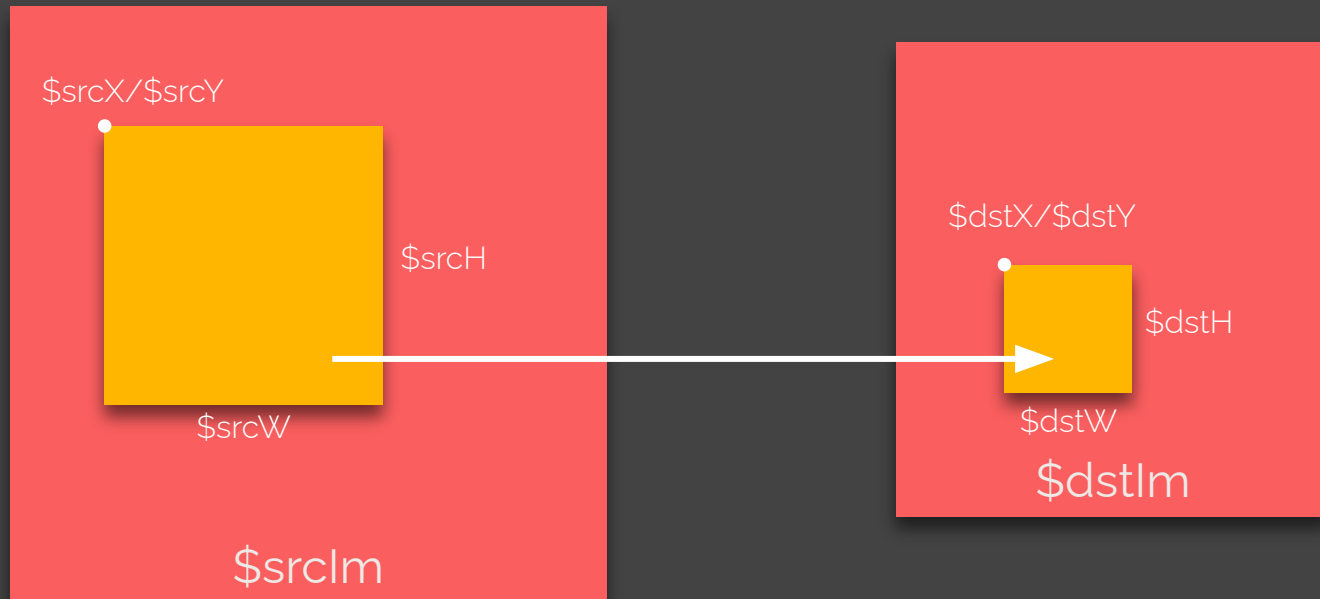
```
header("Content-type: image/png");  
imagepng($image2);
```



Bilder skalieren



```
imagecopyresampled($dstIm, $srcIm, $dstX, $dstY, $srcX, $srcY,  
                  $dstW, $dstH, $srcW, $srcH);
```



Ausschnitt skalieren

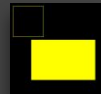
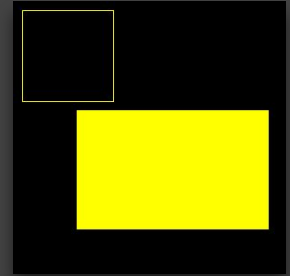


```
$image1 = imagecreatetruecolor(300, 300);  
$image2 = imagecreatetruecolor(100, 100);  
$yellow = imagecolorallocate($image1, 255, 255, 0);
```

```
imagerectangle($image1, 10, 10, 110, 110, $yellow);  
imagefilledrectangle($image1, 70, 120, 280, 250, $yellow);
```

```
imagecopyresampled($image2, $image1, 0, 0, 0, 0, 100, 100, 300, 300);
```

```
header("Content-type: image/png");  
imagepng($image2);
```



Bilder **rotieren**



```
imagerotate($image, $angle, $bgcolor);
```

```
$image = imagecreatetruecolor(300, 300);
```

```
$yellow = imagecolorallocate($image, 255, 255, 0);
```

```
$black = imagecolorallocate($image, 0, 0, 0);
```

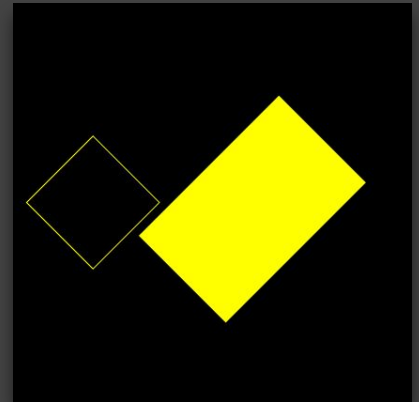
```
imagerectangle($image, 10, 10, 110, 110, $yellow);
```

```
imagefilledrectangle($image, 70, 120, 280, 250, $yellow);
```

```
$image = imagerotate($image, 45, $black);
```

```
header("Content-type: image/png");
```

```
imagepng($image);
```



PDFs erzeugen

Mit Dompdf



Dompdf

PDF-Bibliothek.

Weite Verbreitung (WooCommerce, Magento, CiviCRM, etc.).

<https://github.com/dompdf/dompdf>



Installation

```
$ composer require dompdf/dompdf
```

Einbindung mittels Composer-Autoloader:

```
require "vendor/autoload.php";
```

Initialisierung und Ablauf



```
use Dompdf\Dompdf;  
require __DIR__ . "/vendor/autoload.php";  
  
$dompdf = new Dompdf();  
  
$dompdf->loadHtml("<h1>Hello PDF World!</h1><p>My first PDF with Dompdf!</p>");  
  
$dompdf->setPaper("A4", "portrait");  
  
$dompdf->render();
```

Ausgabe des PDFs



PDF im Browser anzeigen:

```
$dompdf→stream("hallo-welt.pdf", ["Attachment" ⇒ false]);
```

PDF als Datei schreiben:

```
file_put_contents(__DIR__ . "/hallo-welt.pdf", $dompdf→output());
```

Anpassen der Optionen



```
use Dompdf\Dompdf;  
use Dompdf\Options;
```

```
$options = new Options();  
$options→setDefaultFont("Helvetica");
```

```
$dompdf = new Dompdf($options);
```

```
// PDF-Erstellung...
```

Metadaten setzen



Titel:

```
$dompdf→addInfo("Title", "Dompdf Example");
```

Autor*in:

```
$dompdf→addInfo("Author", "Wolfgang Hochleitner");
```

Thema des PDFs:

```
$dompdf→addInfo("Subject", "Dompdf Demo");
```

Schlüsselwörter:

```
$dompdf→addInfo("Keywords", "PHP, PDF, Dompdf, Web Technologies");
```

PDF-Software:

```
$dompdf→addInfo("Creator", "Dompdf v3.1.5");
```

Vollständiges Beispiel



```
$options = new Options();
$options→setDefaultFont("Helvetica");

$dmpdf = new Dmpdf($options);

$dmpdf→loadHtml("<h1>Hello PDF World!</h1><p>My first PDF with Dmpdf!</p>");
$dmpdf→setPaper("A4", "portrait");

$dmpdf→render();

$dmpdf→addInfo("Title", "Dmpdf Example");
$dmpdf→addInfo("Author", "Wolfgang Hochleitner");
$dmpdf→addInfo("Subject", "Dmpdf Demo");
$dmpdf→addInfo("Keywords", "PHP, PDF, Dmpdf, Web Technologies");
$dmpdf→addInfo("Creator", "Dmpdf v3.1.5");

$dmpdf→stream("hello-world.pdf", ["Attachment" ⇒ false]);

file_put_contents(__DIR__ . "/hello-world.pdf", $dmpdf→output());
```

Dompdf & Latte



PDFs mit einer Template-Engine erstellen

1. Daten aus PHP
2. Latte fügt Daten in ein Template ein, rendert HTML als String
3. Dompdf wandelt den String in ein PDF um.



Installation

```
$ composer require dompdf/dompdf latte/latte
```

Einbindung mittels Composer-Autoloader:

```
require "vendor/autoload.php";
```

Das Latte- Template



```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <title>{$title}</title>
  <style> ←!— Styles → </style>
</head>
<body>
  ←!-- Seiteninhalt →
</body>
<html>
```

Styles (in <style>)



```
body {  
  font-family: Helvetica, Arial, sans-serif;  
}  
table {  
  border-collapse: collapse;  
  width: 100%;  
  margin-top: 20px;  
}  
th, td {  
  border: 1px solid #333;  
  padding: 8px;  
  text-align: left;  
}  
th {  
  background-color: #f2f2f2;  
}
```

Seiteninhalt (in <body>)



```
<h1>{$title}</h1>

<div class="image">
  
</div>

<p>Übersicht der Vorlesungsinhalte:</p>

<table>
  <tr>
    <th>Einheit</th>
    <th>Thema</th>
  </tr>
  <tr n:foreach="$lectures as $unit ⇒ $topic">
    <td>{$unit}</td>
    <td>{$topic}</td>
  </tr>
</table>
```

PHP-Logik Teil 1



```
// Latte initialisieren
$latte = new Engine();
$latte->setLoader(new FileLoader(__DIR__ . "/templates"));
$latte->setCacheDirectory(__DIR__ . "/cache");

// Daten fürs Template
$data = [
    "title" => "Willkommen zu Web Technologies",
    "lectures" => [
        "Vorlesung 1" => "Grundlagen der Kommunikation im Web",
        "Vorlesung 2" => "PHP-Grundlagen Teil 1",
        // ...
    ],
];

// Template in einen String rendern
$html = $latte->renderToString("webtechnologies.latte", $data);
```

PHP-Logik Teil 2



```
// Dompdf mit Optionen erzeugen
$options = new Options();
$options->setChroot(__DIR__);
$dompdf = new Dompdf($options);

$dompdf->loadHtml($html);
$dompdf->setPaper("A4", "portrait");
$dompdf->render();

// Metadaten hinzufügen
$dompdf->addInfo("Title", "Dompdf & Latte Example");
$dompdf->addInfo("Author", "Wolfgang Hochleitner");
$dompdf->addInfo("Subject", "Dompdf & Latte Demo");
$dompdf->addInfo("Keywords", "PHP, PDF, Dompdf, Latte, Web Technologies");
$dompdf->addInfo("Creator", "Dompdf v3.1.5");

// PDF anzeigen und speichern
$dompdf->stream("pdf-latte-webtechnologies.pdf", ["Attachment" => false]);
file_put_contents(__DIR__ . "/pdf-latte-webtechnologies.pdf", $dompdf->output());
```



Nächstes Mal in WET2VO

REST APIs & AJAX

Credits



- S. 3: Buzz has both feet on the footpad. His hands are between the third and fourth rungs. His bent knees suggest that he is about to try to jump up to the lowest rung. His OPS antenna is up: <https://history.nasa.gov/alsj/a11/images11.html#Mission>
- S. 22: sabre/xml Social Image: <https://github.com/sabre-io/xml>
- S. 36: gdlb Logo: <https://libgd.github.io/>
- S. 53 & 60: Dompdf Logo: <https://github.com/dompdf/dompdf>
- S. 60: Latte Logo: <https://latte.nette.org/>